



PROJECT REPORT

AI Based Resume Analyzer

Submitted By: Akshat Bajpai

Date: September 18, 2026

Version: 1.0

Table of Contents

Section	Topic	Page No.
1	Cover Page	1
-	Table of Contents	2
2	Introduction	3
3	Problem Statement	3
4	Functional Requirements	4
5	Non-functional Requirements	4
6	System Architecture	5
7	Design Diagrams	5
8	Design Decisions & Rationale	6
9	Implementation Details	6
10	Screenshots / Results	7
11	Testing Approach	7
12	Challenges Faced	8
13	Learnings & Key Takeaways	8
14	Future Enhancements	9
15	References	9

2. Introduction

In today's business world human resources departments receive thousands of applications for one job posting. Screening these resumes by hand takes a lot of time can be biased and is not efficient. The AI Based Resume Analyzer uses intelligence and Natural Language Processing to automate the resume screening process. It pulls out details such, as skills, education and experience then compares those details with the job description. The AI Based Resume Analyzer then ranks candidates according to how they fit the job. Recruiters can use the AI Based Resume Analyzer to make objective hiring decisions and find top talent in a large pool of applicants.**3.**

3. Problem Statement

The usual way of hiring people has a problem when it comes to checking the resumes. HR people usually look at a resume for 6 to 7 seconds. This quick look makes it easy to have mistakes to be biased and to miss good people. Also looking for words in the resume doesn't work well because it doesn't understand the real meaning. For example someone who knows about 'machine learning' might not be found if the job is looking for 'predictive modeling'. There is a need for a system that can really understand what the resume and the job description mean so it can give the best people quickly and fairly.

4. Functional Requirements

The system needs to do the following things to help the people who use it (recruiters and HR managers):

- Document Parsing: Let people upload and take the text out of resume types like PDF, DOCX, TXT.
- Entity Extraction: Use language models to find parts of the resume like name, contact info, skills, education and work history.
- Job Description (JD) Input: Let recruiters put in the job details or upload them.
- Semantic Matching Engine: Find how similar the resume is to the job description using numbers that show meaning.
- Candidate Ranking: Put the people in order based on how they match the job in a single place.

- **Feedback Loop:** Let recruiters change the scores or give comments to help the system get better.
- **Data Export:** Let people take the list of candidates and put it into a file, like CSV or Excel.

5. Non-functional Requirements

- **Performance:** The system needs to handle and rate a group of 100 resumes in than 30 seconds.
- **Scalability:** The system is built using microservices so that it can manage traffic when many people are being hired at the same time.
- **Security & Privacy:** The system follows all the rules of GDPR and CCPA. Resumes have details so the data must be safe when stored (using AES-256) and when moving between places (using TLS 1.2 or higher).
- **Availability:** The system must be working 99.9% of the time. It is placed on cloud systems with load balancing to make sure it stays up.
- **Usability:** The system has a simple and easy to use interface. New HR staff should be able to learn how to use it in under 30 minutes.

6. System Architecture

The system uses a three-layer cloud-based design with a presentation part, an application and AI part and a data storage part. The front end is made with React.js. It talks to the back end using RESTful APIs. The back end is built with FastAPI (Python). The back end handles requests. Sends documents to the NLP Engine. The NLP Engine is based on HuggingFace Transformers and spaCy. The details and meaning of the text are saved in a PostgreSQL database and a Vector Database, like Pinecone or Milvus to make finding items faster.

7. Design Diagrams

Note: The structural definitions of the design diagrams are outlined below.

Use Case Diagram

- Actors: Recruiter, System Administrator.
- Use Cases: Upload Resumes, Input JD, View Rankings, Download Report, Manage Users, Train/Fine-tune Model.

Workflow Diagram

Start -> Recruiter Uploads Resumes & JD -> System validates file types -> OCR/Text Extraction -> NLP Pipeline (NER & Tokenization) -> Embedding Generation -> Cosine Similarity Calculation -> System Generates Ranked List -> End.

Sequence Diagram

1. Frontend -> Sends File to Backend via POST API.
2. Backend -> Requests Parsing from NLP Microservice.
3. NLP Service -> Returns Parsed JSON Data.
4. Backend -> Sends parsed data and JD to Matching Service.
5. Matching Service -> Queries Vector DB for similarity -> Returns Score.
6. Backend -> Stores result in PostgreSQL -> Returns 200 OK + JSON payload to Frontend.

Class / Component Diagram

- Components: UI_Dashboard, Resume_Ingestion_Service, Text_Parser, NER_Model, Matcher_Engine, Database_Manager.
- Classes: Candidate, JobDescription, MatchScore, ParserUtility.

ER Diagram (Storage Design)

Entities:

- JOB (Job_ID [PK], Title, Description, Date_Posted)
- CANDIDATE (Candidate_ID [PK], Name, Email, Phone, Resume_URL)
- EXTRACTED_SKILLS (Skill_ID [PK], Candidate_ID [FK], Skill_Name)
- APPLICATION_SCORE (Score_ID [PK], Job_ID [FK], Candidate_ID [FK], Match_Percentage)

8. Design Decisions & Rationale

1. Deep Learning over TF-IDF: We picked transformer-based models such as BERT over traditional TF-IDF because BERT understands context in both directions. This means it can correctly match job titles like 'Software Engineer' and 'Backend Developer' even if they are not the same words.

2. FastAPI over Django: We went with FastAPI for the backend because it supports operations natively. This is especially helpful when handling tasks that involve input or output operations, such as reading PDF files or sending requests to machine learning models.

3. Vector Database: Using a vector database helps us quickly find the matches when comparing resume embeddings, with job description embeddings. This makes the matching process fast. Reduces delays ensuring low latency in results.

9. Implementation Details

Technology Stack:

- Frontend: React.js, Tailwind CSS
- Backend: Python, FastAPI
- NLP Models: spaCy (en_core_web_trf) for Named Entity Recognition, HuggingFace SentenceTransformers (all-MiniLM-L6-v2) for embeddings.
- Database: PostgreSQL (Relational Data), Pinecone (Vector Data)
- Deployment: Docker, AWS ECS, S3 for Resume storage.

Core Algorithms:

The text extraction relies on PyMuPDF and pdfplumber. Once text is extracted, regex and spaCy extract structured fields. The parsed text is fed into the SentenceTransformer to output a dense 384-dimensional vector. The JD is processed similarly. We calculate the Cosine Similarity between the candidate vector and JD vector to produce a matching percentage from 0 to 100%.

10. Screenshots / Results

[Placeholder for User Interface Screenshots]

- Screenshot 1: The main dashboard showing a bar chart of top 5 candidates.
- Screenshot 2: The resume upload portal with drag-and-drop functionality.
- Screenshot 3: A detailed candidate profile view highlighting matched skills in green and missing skills in red.

11. Testing Approach

The system underwent rigorous testing at multiple levels:

- Unit Testing: pytest was used to test individual functions, particularly the text extraction and regex pattern matching for emails and phone numbers.
- Integration Testing: Testing API endpoints using Postman to ensure seamless communication between the React frontend and FastAPI backend.
- Accuracy Testing (AI): A golden dataset of 500 manually annotated resumes was used to measure precision, recall, and F1-score of the NER model. The system achieved a 92% F1-score on skill extraction.

- Load Testing: Apache JMeter was utilized to simulate 50 concurrent HR users uploading batches of resumes simultaneously to verify system stability.

12. Challenges Faced

1. Resume Formats: Candidates create many creative layouts with multiple columns and tables. These designs interfere with the top-to-bottom reading of text. To handle this we used layout-aware parsers, such as pdfplumber to fix the issue.

2. Image-Based PDFs: A lot of resumes are scanned images, not text PDFs. We added Tesseract OCR as a tool for reading these images, which made processing a little slower.

3. AI Bias: We had to make sure the model did not naturally favor groups, such as, by names or university locations. This required careful selection of training data. Using bias-mitigation methods.

13. Learnings & Key Takeaways

- NLP limitations: We learned that out-of-the-box NER models are not perfectly suited for domain-specific HR data and require fine-tuning on domain corpora.

- User Experience is Key: AI output must be interpretable. Providing a simple percentage score wasn't enough for recruiters; they needed "explainable AI" highlighting exactly WHY a candidate scored highly.

14. Future Enhancements

1. ATS Integration: Connecting the analyzer with Application Tracking Systems such as Workday, Greenhouse and SAP SuccessFactors in a smooth way.

2. Automated Interview Scheduling: Linking with Google Calendar and Outlook APIs to automatically send invitations to the candidates for technical tests.

3. Web Footprint Analysis: Including options to collect and look at public GitHub projects or LinkedIn pages to check a candidates real skills.

4. Conversational Chatbot: A chat system powered by AI where recruiters can say: Show me the Python developers, with AWS experience.

15. References

1. Devlin, J., et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

2. spaCy Documentation. Industrial-strength Natural Language Processing in Python.
<https://spacy.io/>
3. Hugging Face Sentence Transformers. <https://sbert.net/>
4. FastAPI Documentation. High-performance Python web framework.
<https://fastapi.tiangolo.com/>